

Відокремлений структурний підрозділ
«Волинський фаховий коледж Національного університету харчових
технологій»

Відділення Інформатики та комп'ютерної техніки
Циклова випускова комісія комп'ютерної та програмної інженерії

ЗВІТ-ЩОДЕННИК навчальної практики

Тема проєкту: «Менеджер подій»
Варіант 27

Спеціальність: 121/F2 Інженерія програмного забезпечення
Освітньо-професійна програма: Інженерія програмного забезпечення
Освітньо-професійний ступінь: фаховий молодший бакалавр

Здобувач освіти II курсу, групи Трефіловський Даниїл Дмитрович
Група: ІПЗ-23-2

Керівник практики від коледжу: Шепелюк Галина Степанівна

Репозиторій: <https://github.com/mistmandanoxi-commits/practice>

Луцьк 2026

Календарний графік проходження практики

№	Назви робіт	Тиждень 1	Тиждень 2
1	Вступний інструктаж з техніки безпеки, протипожежної безпеки та виробничої санітарії	15.06	
2	Формулювання технічного завдання	15.06 - 16.06	
3	Реалізація програмного продукту	17.06 - 19.06	22.06
4	Тестування програмного продукту		23.06 - 24.06
5	Оформлення звіту-щоденника практики		25.06 - 26.06

Робочі записи здобувача освіти під час практики

Дата	Короткий опис діяльності
15.06.26	Ознайомився з програмою навчальної практики, вимогами до виконання та оформлення звіту. Отримав індивідуальне завдання за варіантом №27 «Менеджер подій».
16.06.26	Провів аналіз предметної області. Сформував технічне завдання майбутнього програмного продукту, визначив функціональні та нефункціональні вимоги.
17.06.26	Побудував UML діаграму варіантів використання (Use Case). Визначив основні сценарії взаємодії користувача із системою.
18.06.26	Спроектував структуру програми та структуру даних для зберігання інформації про події.
19.06.26	Реалізував функцію додавання нових подій та запису інформації у файл.
22.06.26	Реалізував завантаження даних із файлу та відображення збережених подій.
23.06.26	Реалізував функції перегляду майбутніх і минулих подій на основі аналізу дат.
24.06.26	Реалізував функцію пошуку подій за назвою. Додав перевірку коректності введення користувача.
25.06.26	Провів тестування програмного продукту. Перевірив усі основні функції та сценарії використання. Підготував UML-діаграму, скріншоти роботи програми та оформив основну частину звіту.
26.06.26	Завершив звіт-щоденник, перевінив запуск проєкту. Підготував архів для завантаження на GitHub.

Зміст звіту практики

1 Технічне завдання

- 1.1 Призначення системи
- 1.2 Функціональні вимоги
- 1.3 Нефункціональні вимоги
- 1.4 Формат збереження даних
- 1.5 Обмеження системи
- 2 Діаграма варіантів використання
 - 2.1 Актори системи
 - 2.2 UML use case діаграма
 - 2.3 Опис варіантів використання
- 3 Опис реалізації
 - 3.1 Структура проєкту
 - 3.2 Опис функцій
 - 3.3 Особливості реалізації
 - 3.4 Скріншоти застосунку
- 4 Код програми
- 5 Тестування програмного продукту
 - 5.1 Методика тестування
 - 5.2 Таблиця тест-кейсів
- 6 Автоматизований тест
- Висновки

1 Технічне завдання

1.1 Призначення системи

Програма «Менеджер подій» призначена для зберігання та керування інформацією про події. Користувач може додавати нові події, переглядати майбутні та минулі події, а також виконувати пошук за назвою.

1.2 Функціональні вимоги

Система реалізує основні функції, наведені в таблиці 1.1.

№	Назва функції	Опис	Вхідні дані
1	Додавання нової події	Запитує назву події, дату проведення та місце проведення. Зберігає нову подію у файл.	Назва події, дата, місце проведення
2	Перегляд майбутніх подій	Виводить список усіх подій, дата яких є більшою або дорівнює поточній даті.	—
3	Перегляд минулих подій	Виводить список усіх подій, дата яких є меншою за поточну дату.	—
4	Пошук події за назвою	Дає можливість знайти подію за повною або частковою назвою.	Назва події
5	Збереження даних	Автоматично записує інформацію про події до текстового файлу після додавання нової події.	Дані події
6	Завантаження даних	Зчитує інформацію з файлу під час запуску програми та формує список подій.	Файл events.txt
7	Перевірка коректності введення	Контролює правильність введення пунктів меню та запобігає аварійному завершенню програми.	Дані користувача
8	Завершення роботи програми	Коректно завершує роботу застосунку після вибору відповідного пункту меню.	Команда виходу

1.3 Нефункціональні вимоги

Мова реалізації: C++17.

Для розробки використано стандартні бібліотеки мови C++: `iostream`, `vector`, `fstream`, `sstream`, `string`, `ctime`, `limits`.

Зовнішні бібліотеки не використовуються.

Дані зберігаються у текстовому файлі **events.txt** у кодуванні UTF-8.

Програма має консольний інтерфейс користувача.

Некоректний ввід не призводить до аварійного завершення програми. У разі помилки користувач отримує відповідне повідомлення та повертається до головного меню.

Дані про події автоматично завантажуються з файлу під час запуску програми та автоматично зберігаються після додавання нової події.

Програма може бути скомпільована та запущена в будь-якому середовищі розробки, що підтримує стандарт C++17 (Visual Studio, Code::Blocks, Dev-C++, VS Code, OnlineGDB).

Для запуску програми необхідно скомпілювати файл main.cpp та виконати отриманий виконуваний файл.

1.4 Формат збереження даних

Інформація про події зберігається у текстовому файлі **events.txt**.

Кожен рядок файлу містить одну подію та складається з трьох полів, розділених символом |:

Концерт|2026-07-15|Львів

Конференція|2026-08-10|Київ

Зустріч випускників|2025-05-20|Луцьк

1.5 Обмеження системи

Редагування існуючих подій не реалізовано.

Видалення подій не реалізовано.

Програма працює лише з локальним файлом events.txt.

Дата події повинна вводитися у форматі РРРР-ММ-ДД.

Система підтримує одного користувача та не використовує мережеве підключення.

2 Діаграма варіантів використання

2.1 Актори системи

Система має одного актора — Користувача.

Користувач є основним учасником взаємодії із системою та працює з програмою через консольний інтерфейс. Він може додавати нові події, переглядати майбутні та минулі події, виконувати пошук подій за назвою та завершувати роботу програми.

Усі функції системи виконуються безпосередньо за ініціативою користувача через головне меню програми.

2.2 UML use case діаграма

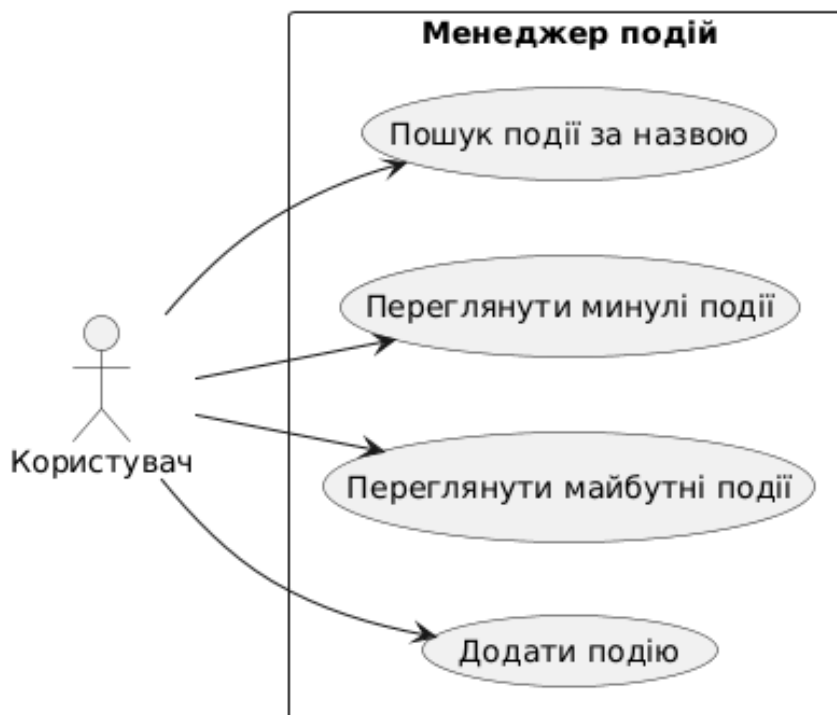


Рисунок 2.1 UML-діаграма варіантів використання системи «Менеджер подій»

2.3 Опис варіантів використання

UC	Назва	Основний сценарій	Альтернативний сценарій
----	-------	-------------------	-------------------------

1	Додати подію	Користувач вводить назву події, дату проведення та місце проведення. Система зберігає подію у файл та додає її до списку.	Якщо будь-яке поле порожнє або дата введена некоректно, подія не зберігається і виводиться повідомлення про помилку.
2	Переглянути майбутні події	Користувач обирає відповідний пункт меню. Система виводить список подій, дата яких ще не настала.	Якщо майбутніх подій немає, виводиться повідомлення «Майбутніх подій немає».
3	Переглянути минулі події	Користувач обирає відповідний пункт меню. Система відображає список подій, дата яких уже минула.	Якщо минулих подій немає, виводиться повідомлення «Минулих подій немає».
4	Пошук події за назвою	Користувач вводить назву або її частину. Система виконує пошук та відображає знайдені події.	Якщо збігів не знайдено, виводиться повідомлення «Подію не знайдено».
5	Завантаження даних	Під час запуску програми система автоматично зчитує інформацію з файлу events.txt та формує список подій.	Якщо файл відсутній або порожній, програма запускається з порожнім списком подій.
6	Збереження даних	Після додавання нової події система автоматично записує всі дані до файлу events.txt.	Якщо виникає помилка відкриття файлу, користувач отримує повідомлення про помилку збереження.
7	Обробка помилкового введення	Користувач вводить номер пункту меню. Система перевіряє коректність введених даних.	Якщо введено неправильний номер або нечислове значення, система повідомляє про помилку та повертає користувача до меню.
8	Завершення роботи програми	Користувач обирає пункт «Вихід». Система завершує виконання програми.	Якщо користувач не обирає вихід, програма продовжує роботу та повертається до головного меню.

3 Опис реалізації

3.1 Структура проєкту

```
EventManager/  
├─ main.cpp  
├─ events.txt  
├─ README.md  
├─ use_case_diagram.png  
└─ screenshots
```

3.2 Опис функцій

loadFromFile() — завантаження даних.

saveToFile() — збереження даних.

addEvent() — додавання події.

showFutureEvents() — майбутні події.

showPastEvents() — минулі події.

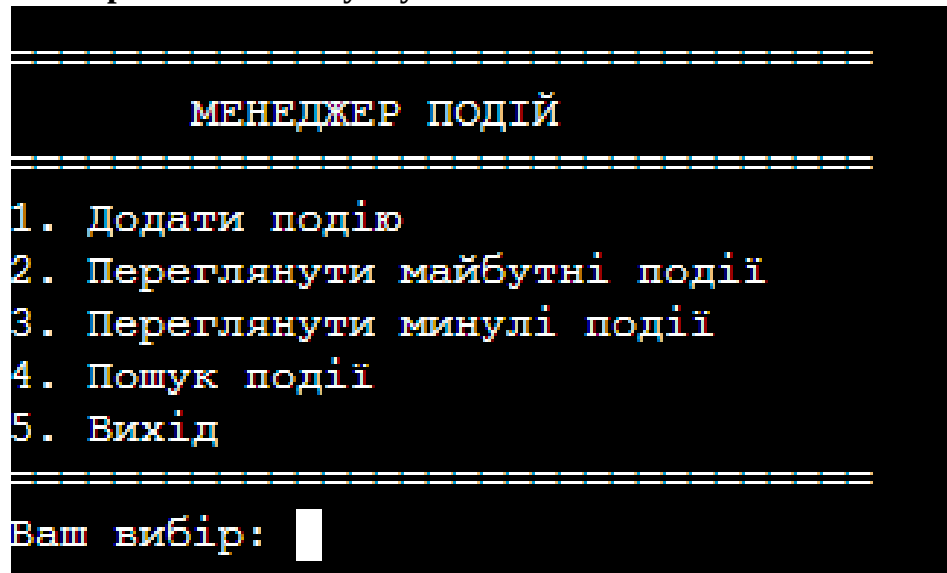
searchEvent() — пошук події.

menu() — головне меню.

3.3 Особливості реалізації

Програма використовує структуру Event та контейнер vector для зберігання інформації про події. Дані автоматично зберігаються у файл events.txt і завантажуються при запуску програми.

3.4 Скріншоти застосунку



```
=====
МЕНЕДЖЕР ПОДІЙ
=====
1. Додати подію
2. Переглянути майбутні події
3. Переглянути минулі події
4. Пошук події
5. Вихід
=====
Ваш вибір: █
```


Назва події: Концерт Брітні Спірс
Дата (YYYY-MM-DD): 2026-08-28
Місце проведення: Paris, France

Подію успішно додано!

===== Майбутні події =====

Назва: Концерт Брітні Спірс
Дата: 2028-08-12
Місце: New York, USA

Назва: Концерт Брітні Спірс
Дата: 2026-08-28
Місце: Paris, France

===== Минулі події =====

Назва: Кон?вис?сBritney Spears' VMAs
Дата: 2007-09-05
Місце: Los-Angeles, USA

Назва: VMAs 2007
Дата: 2007-09-05
Місце: Los-Angeles, USA

Введіть назву події: 2007

Назва: VMA's 2007

Дата: 2007-09-05

Місце: Los-Angeles, USA

4 Код програми

Нижче наведено повний вихідний код файлу main.cpp.

```
#include <iostream>
#include <vector>
#include <fstream>
#include <sstream>
#include <string>
#include <ctime>
#include <limits>

using namespace std;

struct Event
{
    string name;
    string date;
    string place;
};

vector<Event> events;
const string FILE_NAME = "events.txt";

void saveToFile()
{
    ofstream file(FILE_NAME);

    for (const auto& e : events)
    {
        file << e.name << "|"
              << e.date << "|"
              << e.place << endl;
    }

    file.close();
}

void loadFromFile()
{
    ifstream file(FILE_NAME);

    if (!file)
        return;

    string line;
```

```

while (getline(file, line))
{
    stringstream ss(line);
    Event e;

    getline(ss, e.name, '|');
    getline(ss, e.date, '|');
    getline(ss, e.place, '|');

    events.push_back(e);
}

file.close();
}

bool isFutureDate(const string& date)
{
    time_t now = time(0);
    tm* current = localtime(&now);

    int currentYear = current->tm_year + 1900;
    int currentMonth = current->tm_mon + 1;
    int currentDay = current->tm_mday;

    int year, month, day;

    if (sscanf(date.c_str(), "%d-%d-%d",
               &year, &month, &day) != 3)
    {
        return false;
    }

    if (year > currentYear)
        return true;

    if (year == currentYear && month > currentMonth)
        return true;

    if (year == currentYear &&
        month == currentMonth &&
        day >= currentDay)
        return true;

    return false;
}

void addEvent()
{
    Event e;

    cin.ignore();

    cout << "\nНазва події: ";
    getline(cin, e.name);

    cout << "Дата (YYYY-MM-DD): ";
    getline(cin, e.date);

    cout << "Місце проведення: ";
    getline(cin, e.place);

    events.push_back(e);

    saveToFile();

    cout << "\nПодію успішно додано!\n";
}

```

```

void showFutureEvents()
{
    cout << "\n===== Майбутні події =====\n";

    bool found = false;

    for (const auto& e : events)
    {
        if (isFutureDate(e.date))
        {
            cout << "\nНазва: " << e.name
                << "\nДата: " << e.date
                << "\nМісце: " << e.place
                << "\n-----\n";

            found = true;
        }
    }

    if (!found)
    {
        cout << "Майбутніх подій немає.\n";
    }
}

void showPastEvents()
{
    cout << "\n===== Минулі події =====\n";

    bool found = false;

    for (const auto& e : events)
    {
        if (!isFutureDate(e.date))
        {
            cout << "\nНазва: " << e.name
                << "\nДата: " << e.date
                << "\nМісце: " << e.place
                << "\n-----\n";

            found = true;
        }
    }

    if (!found)
    {
        cout << "Минулих подій немає.\n";
    }
}

void searchEvent()
{
    cin.ignore();

    string keyword;

    cout << "\nВведіть назву події: ";
    getline(cin, keyword);

    bool found = false;

    for (const auto& e : events)
    {
        if (e.name.find(keyword) != string::npos)
        {
            cout << "\nНазва: " << e.name
                << "\nДата: " << e.date
                << "\nМісце: " << e.place
                << "\n-----\n";
        }
    }
}

```

```

        found = true;
    }
}

if (!found)
{
    cout << "\nПодію не знайдено.\n";
}

}

void menu()
{
    cout << "\n=====";
    cout << "\n        МЕНЕДЖЕР ПОДІЙ";
    cout << "\n=====";
    cout << "\n1. Додати подію";
    cout << "\n2. Переглянути майбутні події";
    cout << "\n3. Переглянути минулі події";
    cout << "\n4. Пошук події";
    cout << "\n5. Вихід";
    cout << "\n=====";
    cout << "\nВаш вибір: ";
}

int main()
{
    loadFromFile();

    int choice;

    do
    {
        menu();

        while (!(cin >> choice))
        {
            cout << "\nПомилка! Введіть число: ";

            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
        }

        switch (choice)
        {
            case 1:
                addEvent();
                break;

            case 2:
                showFutureEvents();
                break;

            case 3:
                showPastEvents();
                break;

            case 4:
                searchEvent();
                break;

            case 5:
                cout << "\nРоботу завершено.\n";
                break;

            default:
                cout << "\nНевірний пункт меню!\n";
        }
    }
}

```

```
    } while (choice != 5);  
    return 0;  
}
```

5 Тестування програмного продукту

5.1 Методика тестування

Тестування проводилось методом ручного запуску утиліти з виконанням штатних і нештатних сценаріїв. Кожен тест перевіряє окрему функцію програми. Перед частиною тестів файл data.json очищався, щоб результати не залежали від попередніх запусків.

5.2 Таблиця тест-кейсів

№	Сценарій	Очікуваний результат	Статус
1	Додавання події	Подія додається	Пройдено
2	Перегляд майбутніх подій	Події відображаються	Пройдено
3	Перегляд минулих подій	Події відображаються	Пройдено
4	Пошук існуючої події	Подія знайдена	Пройдено
5	Пошук неіснуючої події	Виведено повідомлення	Пройдено
6	Перевірка файлу	Дані зберігаються	Пройдено
7	Некоректний пункт меню	Помилка обробляється	Пройдено
6	Перевірка файлу	Дані зберігаються	Пройдено
7	Некоректний пункт меню	Помилка обробляється	Пройдено

Висновки

Під час проходження навчальної практики було розроблено консольну програму «Менеджер подій» мовою програмування C++. У програмі реалізовано додавання подій, перегляд майбутніх та минулих подій, пошук за назвою та збереження інформації у файл.

У ході виконання роботи було закріплено навички роботи зі структурами даних, файлами, контейнерами STL та функціями мови C++. Усі поставлені завдання виконано повністю, а програма відповідає вимогам індивідуального варіанту №27.

У майбутньому утиліту можна розширити: додати редагування книги, пошук за назвою, експорт статистики у CSV, розподіл на модулі storage.py, diary.py і cli.py, а також побудову графіків читання. <3